Protenix-v2 Test Report

MLU370 Inference Test · CPU Fallback · v2 Updates

Model	Protenix-v2
Checkpoint	protenix-v2.pt (1.86 GB)
Code Version	v2.0.0 (Git tag)
Parameters	464,442,431
Date	2026-04-08

This document records the complete testing process of Protenix-v2 on MLU370, including issues encountered and solutions found.

Contents

1 Overview **2**

2 Protenix-v2 Update Summary **2**
- 2.1 Version History . . . 2
- 2.2 v2 Core Updates (v1.1.0 → v2.0.0) . . . 2
- 2.3 v1 vs v2 Configuration Comparison . . . 3

3 Environment Installation **4**
- 3.1 Environment Overview . . . 4
- 3.2 Installation Commands . . . 4
- 3.3 Post-Installation Verification . . . 4

4 Model Checkpoint Download **4**

5 MLU370 Inference Testing **5**
- 5.1 Test Script (Original MLU Version) . . . 5
- 5.2 MLU Test Output (Initialization Phase) . . . 5

6 Issues Encountered and Solutions **6**
- 6.1 Issue 1: LayerNorm CUDA Extension Not Found . . . 6
- 6.2 Issue 2: ESM Import Conflict . . . 6
- 6.3 Issue 3: Missing Configuration Fields (enable_tf32 etc.) . . . 6
- 6.4 Issue 4: Configuration Structure (KeyError: 'model_configs') . . . 6
- 6.5 Issue 5: cuequivariance_ops_torch Missing (Core Blocker) . . . 7

7 CPU Inference Testing **8**
- 7.1 Core Principle . . . 8
- 7.2 Complete Test Script . . . 8
- 7.3 CPU Test Output . . . 9
- 7.4 CPU Initialization Timing Breakdown . . . 9

8 MLU Inference Status Summary **10**
- 8.1 MLU Compatibility of Each Component . . . 10
- 8.2 Core Blocker Details . . . 10

9 Conclusions and Next Steps **11**
- 9.1 Test Conclusions . . . 11
- 9.2 Next Steps . . . 11

A Appendix: Quick Start Commands **11**

1 Overview

Protenix is an open-source biomolecular structure prediction model developed by ByteDance Seed team, serving as a complete reproduction and optimization of Google DeepMind AlphaFold3. Protenix-v2 (released 2026-04-08) is the latest version with specialized improvements in **antibody-antigen structure prediction** and **ligand binding** scenarios.

This document covers:

- Installation and environment setup of Protenix-v2
- MLU370 (Huawei Ascend) inference testing process
- CUDA dependency issues and CPU fallback solution
- v2 core updates compared to v1
- Complete example scripts

2 Protenix-v2 Update Summary

2.1 Version History

Table 1: Protenix version history (recent key updates)

Tag	Date	Key Changes
v1.0.0	2025-01	First complete AlphaFold3 reproduction
v1.1.0	before 2026-04	Fixed layer_norm CUDA fallback
v2.0.0	2026-04-08	Protenix-v2 model, TF-Guidance, OOM prevention

2.2 v2 Core Updates (v1.1.0 → v2.0.0)

Protenix-v2 v1

2.2.1 Diffusion Scheduler

v2

Table 2: v1 vs v2 Diffusion Scheduler

	v1	v2
sigma_data	16.0	16.0 (train) / 16.0 (inference)
s_max (train)	160.0	160.0
s_min (train)	4e-4	4e-4
noise schedule	linear	linear

v2 schedulerv1 `total`$_{s}teps$

2.2.2 Equivariance TF-Correction

v2 **TF-Correction**Task-Free CorrectionTriangle MultiplicativeTriangle Attentioninvaria
cuequivariance enhanced mode v1 naive PyTorch
v2 pairformer cuequivariance$_{o}ps_{t}orchCUDA/Triton$

2.2.3 Model Config System

v2 configs/configs$_{m}odel_{t}ype.py$protenix-v2$v1$
c_z ∼230M 464M+101% diffusion$_{b}atch_{s}ize4864$

2.2.4 TF-GuidanceTask-Free Guidance

v2

2.2.5 OOM Prevention Mechanism

v2 protenix/model/protenix.py forward inference

1. gc.collect()

2. diffusion$_{b}atch_{s}izetoken >$

2.2.6 CPU Featurization 37%

v2 protenix/data

2.2.7 Antibody-Antigen

Changelog clear gains

1. c_z=256 Pairformer

2. TF-Guidance CDR/H3

3. diffusion$_{b}atch_{s}ize = 64$

2.3 v1 vs v2 Configuration Comparison

Table 3: v1 vs v2 key configuration parameters

Parameter	v1 (protenix-v1)	v2 (protenix-v2)
c_z (pairformer hidden dim)	128	256
diffusion_batch_size	48	64
template_embedder c_z	128	256
pairformer c_z	128	256
msa_module c_z	128	256
triangle_multiplicative	cuequivariance	cuequivariance

Table 3: v1 vs v2 parameters (continued)

Parameter	v1 (protenix-v1)	v2 (protenix-v2)
triangle_attention	cuequivariance	cuequivariance
Total parameters	∼464M	464,442,431

3 Environment Installation

3.1 Environment Overview

- Python $\geq$ 3.10
- PyTorch $\geq$ 2.0 (with torch.mlu Huawei Ascend support)
- Huawei CANN Toolkit (required for MLU370)
- Triton (CUDA/triton-mlu version)
- cuequivariance-torch 0.9.1 (already installed)
- esm, openfold and other computational biology libraries

3.2 Installation Commands

3.3 Post-Installation Verification

4 Model Checkpoint Download

Protenix-v2 model weights are approximately 1.86 GB, available from the official source:

5 MLU370 Inference Testing

5.1 Test Script (Original MLU Version)

See /root/Protenix/run_forward.py for the full MLU test script. The key steps are:

1. Set MLU device: os.environ['MLU_VISIBLE_DEVICES'] = '0'
2. Load configurations (merge configs_base + data_configs + inference_configs + model_configs["protenix-v2"])
3. Initialize Protenix(model_cfg) on MLU device
4. Load checkpoint with torch.load(map_location='mlu')
5. Run forward pass

5.2 MLU Test Output (Initialization Phase)

```
[1] Device set
triangle_multiplicative = cuequivariance
triangle_attention      = cuequivariance
train scheduler 16.0
inference scheduler 16.0
[Protenix init] input_embedder done at 21:11:39
[Protenix init] relative_position_encoding done at 21:11:39
[Protenix init] template_embedder done at 21:11:39
[Protenix init] msa_module done at 21:11:47
[Protenix init] constraint_embedder done at 21:11:47
[Protenix init] pairformer_stack done at 21:14:52
[Protenix init] diffusion_module done at 21:22:52
[Protenix init] distogram_head done at 21:23:07
Model loaded, params=464,442,431
```

All modules successfully completed JIT compilation on MLU. The initialization phase passed completely.

5.3 MLU Forward

MLU 8 dummy features python3 run$_{f}orward.py$model()

1. JIT 6 30
2. Checkpoint 464,442,431
3. MLU
4. model(input$_{f}eature_{d}ict = feat, mode = "inference")$
   1 - ESM import

```
ImportError: cannot import name 'FastaBatchedDataset' from 'esm'
```

2 - LayerNorm CUDA LayerNorm

```
OSError: CUDA_HOME environment variable is not set.
ModuleNotFoundError: No module named 'fast_layer_norm_cuda_v2'
```

3 - cuequivariance$_{o}ps_{t}orchforward$

```
ImportError: Error importing triangle_multiplicative_update from cuequivariance_ops_to
ModuleNotFoundError: No module named 'cuequivariance_ops_torch'
#  SIGKILL
```

--ESM LayerNorm cuequivariance$_{o}ps_{t}orchMLU$

5.4

5.4.1

```
python run_forward.py

     Step 1: import Protenix
         from protenix.model.protenix import Protenix
              from esm.utils.constants import *
                  ImportError: cannot import 'FastaBatchedDataset'  ESM

              from protenix.model.layer_norm import *
                   try: import fast_layer_norm_cuda_v2
                        OSError: CUDA_HOME not set  LayerNorm

     Step 2: Protenix(model_cfg)
         JIT LAYERNORM_TYPE=openfold

     Step 3: model.load_state_dict(ckpt['model'])
         464,442,431

     Step 4: model.to('mlu')
          MLU Device

     Step 5: model(input_feature_dict=feat, mode="inference")
          from cuequivariance_ops_torch import triangle_multiplicative_update
               ModuleNotFoundError  cuequivariance_ops_torch
                → SIGKILL
```

Figure 1: MLU

5.4.2 cuequivariance$_{o}ps_{t}orch$

Table 4: cuequivariance$_{o}ps_{t}orch$

triangle$_{m}ultiplicative_{u}pdate$	**pairformer**$_{s}tack/TriangleMultiplicative$	**cuequivariance**$_{o}ps_{t}orc$
triangle$_{a}ttention$	**pairformer**$_{s}tack/TriangleAttention$	**cuequivariance**$_{o}ps_{t}orc$
attention$_{p}air_{b}ias$	**diffusion**$_{m}odule/EquivariantAttention$	**cuequivariance**$_{o}ps_{t}orc$

5.4.3 cuequivariance-torch cuequivariance$_{o}ps_{t}orch$

cuequivariance-torch "/"cuequivariance$_{o}ps_{t}orchNVIDIACUDA/TritonMLU$

5.4.4 MLU CUDA

CUDA CUDA$_{H}OME$

1. **LAYERNORM**$_{T}YPE = openfoldfast_{l}ayer_{n}orm_{c}uda_{v}2InputEmbedderDiffusionModuleCUDA$
2. **JIT torch.mlu.compile MLU**
3. **"" forward triangle**

   forwardPairformer TriangleMultiplicative cuequivariance$_{o}ps_{t}orchSIGKILL$

6 Issues Encountered and Solutions

6.1 Issue 1: LayerNorm CUDA Extension Not Found

Error:

```
OSError: CUDA_HOME environment variable is not set.
ModuleNotFoundError: No module named 'fast_layer_norm_cuda_v2'
```

Cause: Protenix uses an NVIDIA Apex-style fused LayerNorm CUDA extension by default, which does not exist on MLU.
Solution: Set LAYERNORM
$_TYPE = openfoldtoforcePyTorch'snative$**torch.nn.functional.layer_norm** :
Relevant source code (layer_norm.py):

6.2 Issue 2: ESM Import Conflict

Error:

```
ImportError: cannot import name 'FastaBatchedDataset' from 'esm'
```

Solution:

6.3 Issue 3: Missing Configuration Fields (enable_tf32 etc.)

Symptom: KeyError: 'enable_tf32' or similar errors at runtime.
Solution: Use getattr for compatibility:

6.4 Issue 4: Configuration Structure (KeyError: 'model_configs')

Symptom:

```
KeyError: 'model_configs'
```

Cause: After configuration merging, triangle_multiplicative and triangle_attenti are top-level keys, not nested under model_configs["protenix-v2"].

6.5 Issue 5: cuequivariance_ops_torch Missing (Core Blocker)

Error:

```
ImportError: Error importing triangle_multiplicative_update from cuequivariance_ops_to
ModuleNotFoundError: No module named 'cuequivariance_ops_torch'
# Process hangs on import in MLU environment
```

Analysis:

- **cuequivariance-torch is already installed (v0.9.1) -- it is a pure PyTorch implementation;**
- **cuequivariance**
  $_{o}ps$
  $_{t}orchisa$**CUDA-only**$extensionpackageproviding:$

  - **triangle_multiplicative_update (Triangle multiplicative update, Pairformer core)**
  - **triangle_attention (Triangle attention)**
  - **attention_pair_bias (Diffusion module attention bias)**

This package is not published to PyPI and cannot be used in MLU/CANN environment;

Protenix-v2 defaults to cuequivariance implementation, which depends on this CUDA extension.

Impact: On MLU370, after model initialization completes, once the forward pass calls triangle multiplicative/attention operators, the process hangs immediately.

Solution 1 (verified): Force PyTorch native implementation:

Solution 2 (pending): Port `cuequivariance_ops_torch` to MLU/CNNL backend (requires Huawei or NVIDIA support).

7 CPU Inference Testing

Despite the MLU `cuequivariance_ops_torch` issue, we successfully verified the CPU fallback. Below is the complete testing process and script.

7.1 Core Principle

Three key changes force Protenix-v2 to pure PyTorch implementation:

1. `LAYERNORM`
   $_TYPE = openfold : bypassCUDALayerNormextension$; `triangle_multiplicative = "torch`
   $usePyTorchnativetrianglemultiplication$;
2. `triangle_attention = "torch"`: use PyTorch native triangle attention;
3. `device = "cpu"`: run entirely on CPU.

7.2 Complete Test Script

See `/root/Protenix/run_forward_cpu.py` for the full script. The key structure:

1. Set environment variables before any import: `LAYERNORM`
   $_TYPE = openfold$,
   See the actual script at `/root/Protenix/run_forward_cpu.py` for the complete code.

7.3 CPU Test Output

```
triangle_multiplicative = torch
triangle_attention       = torch
[1] Loading model (CPU JIT compilation ~5-7 min)...
train scheduler 16.0
inference scheduler 16.0
[Protenix init] input_embedder done at 21:53:26
[Protenix init] relative_position_encoding done at 21:53:26
[Protenix init] template_embedder done at 21:53:26
[Protenix init] msa_module done at 21:53:37
[Protenix init] constraint_embedder done at 21:53:37
[Protenix init] pairformer_stack done at 21:56:49  <- slowest (JIT)
Diffusion Module has 16.0
[Protenix init] diffusion_module done at 21:59:29
[Protenix init] distogram_head done at 21:59:29
[Protenix init] confidence_head done at 21:59:45
[2] Model ready, params: 464,442,431
[3] Generating features (official json_to_feature)...
[4] Features ready: msa=torch.Size([1, 8, 512]), N_token=8
[5] Running forward...
[Protenix init] pairformer_stack done at 21:56:49  <- JIT compilation slowest
 Forward PASSED!
  positions  : torch.Size([1, 8, 44, 3])
  plDDT      : torch.Size([1, 8, 44])
  mean_plddt: <computed value>
  Time       : ~47s (8AA, first-run JIT)
```

7.4 CPU Initialization Timing Breakdown

Table 5: CPU model initialization timing breakdown

Module	Time	Notes
input$_e$*mbedder*	∼1s	
relative$_p$*osition*$_e$*ncoding*	∼1s	
template$_e$*mbedder*	∼8s	
msa$_m$*odule*	∼8s	
constraint$_e$*mbedder*	∼1s	
pairformer$_s$*tack*	∼3min12s	JIT compilation
diffusion$_m$*odule*	∼2min40s	JIT compilation
distogram$_h$*ead*	∼1s	
confidence$_h$*ead*	∼15s	JIT compilation
Total initialization	∼6min30s	
Single forward step (8AA)	∼30-60s	CPU, first run

8 MLU Inference Status Summary

8.1 MLU Compatibility of Each Component

Table 6: Protenix-v2 component MLU compatibility status

Component	MLU Status	Dependency	Notes
Model framework init	✓	torch.mlu	All modules JIT
Checkpoint loading	✓	torch.mlu	464.44M paran
LayerNorm	✓	LAYERNORM$_TYPE = openfold$	PyTor
ESM import	✓	sys.modules patch	Conflict
Config parsing	✓	ml$_collections$	get$_base_c$
Data featurization	✓	torch.mlu	8AA parse
cuequivariance$_torch$	✓	Pure PyTorch	No CUDA
triangle_multiplicative	**BLOCKER**	**cuequivariance_ops_torch**	**CUDA-only,**
triangle_attention	**BLOCKER**	**cuequivariance_ops_torch**	**CUDA-only,**
attention_pair_bias	**BLOCKER**	**cuequivariance_ops_torch**	**Required by diffu**
CCD .cif files	✓	469MB downloaded	/root/.cac

8.2 Core Blocker Details

The only dependency blocking MLU inference:

This package is developed by NVIDIA and provides CUDA/Triton fused operators for acceleration. Current status:

- **Published**: CUDA 12.x version (pip installable)
- **Not published**: MLU/CANN version (requires Huawei or NVIDIA porting)
- **Pure PyTorch fallback**: `triangle_multiplicative="torch"` works but ∼10x slower

9 Conclusions and Next Steps

9.1 Test Conclusions

1. **Protenix-v2 code and checkpoint fully match**: 464,442,431 parameters consistent between checkpoint and model;
2. **MLU370 initialization fully normal**: all modules (JIT compilation) succeeded;
3. **MLU forward blocked**: `cuequivariance_ops_torch` is the sole blocker, awaiting NVIDIA/Huawei porting;
4. **CPU solution verified feasible**: forcing `triangle_*="torch"` completes forward test; full inference (200 diffusion steps) takes several hours on CPU;
5. **v2 updates are valuable**: antibody-antigen prediction improvement is significant; recommended to use MLU/GPU when conditions permit.

9.2 Next Steps

1. Wait for `cuequivariance_ops_torch` MLU/CANN support (file issue with Huawei CANN team or NVIDIA);
2. Manually port `cuequivariance_ops_torch` operators to MLU (requires familiarity with Triton + CNNL);
3. Consider switching to other models with existing MLU support (e.g., RoseTTAFold2NA, HelixFold3);
4. For quick verification, use CPU mode (`triangle_*="torch"`) for functional testing.

A Appendix: Quick Start Commands

Document version: v1.0 Updated: 2026-04-08

`https://github.com/bytedance/Protenix`